

Express.js Lecture Notes

1. Web Fundamentals

Before we write backend code, we need to understand how the internet works.

- **Client:** The device or software (like Google Chrome on your laptop, or an app on your phone) that REQUESTS data.
- **Server:** A computer running continuously that listens for requests and PROVIDES data or files back to the client.
- **IP Address:** A unique string of numbers (like 192.168.1.5) that identifies a specific computer on the internet or a local network. Think of it as a house address.
- **DNS (Domain Name System):** Humans can't remember IP addresses easily. DNS acts like the phonebook of the internet. It translates easy-to-read domain names (like 'google.com') into IP addresses.
- **Localhost:** This is the default name for the computer you are currently working on. Its IP address is always 127.0.0.1. When developing locally, we host our servers on localhost.

HTTP Status Codes

When a server sends a response, it attaches a 3-digit status code to summarize what happened:

- **200 OK:** The request was successful.
- **201 Created:** Successful, and a new resource (like a new user) was created.
- **400 Bad Request:** The client sent invalid data.
- **404 Not Found:** The requested URL doesn't exist on the server.
- **500 Internal Server Error:** The server crashed or something went wrong in the backend code.

2. What is Express.js?

Meaning: Express.js is a fast, unopinionated, and minimal web framework built on top of Node.js. While Node.js has a built-in 'http' module to create servers, it requires a lot of manual, messy code to handle routes and data. Express simplifies this process massively, making it the most popular framework for building Node.js REST APIs and web servers.

3. Creating a Basic Server with Express

Let's look at how to create a simple server that handles a GET request.

Code:

```
// 1. Import the express library
const express = require('express');

// 2. Initialize the express application
const app = express();

// 3. Define a GET route for the root URL ( '/')
app.get('/', (req, res) => {
  // res.send() automatically sets the status to 200 and
  Content-Type to HTML
  res.send('<h1>Hello from Express!</h1>');
});

// 4. Start the server on a specific port
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Explanation:

- `const express = require('express');`: Brings the Express package into our file.
- `const app = express();`: Creates an instance of an Express application. The 'app' variable now holds all the methods we need to build our server.
- `app.get(path, callback)`: Tells the server to listen for GET requests at a specific URL path (like '/'). The callback function gives us access to 'req' (request) and 'res' (response).
- `res.send()`: A massive upgrade over Node's raw `res.end()`. It automatically detects what kind of data you are sending (String, JSON, HTML) and sets the correct headers for you.
- `app.listen(port)`: Turns the server on, keeping it awake to listen for incoming traffic on port 3000.

4. Getting Data from the URL

Clients often send data via the URL. Express gives us two main ways to extract this data: Route Parameters and Query Parameters.

Route Parameters (`req.params`)

Meaning: Used to capture specific values exactly where they sit in the URL path. Denoted by a colon (:) in the route definition.

Code:

```
// URL accessed by user: /users/42
app.get('/users/:id', (req, res) => {
  const userId = req.params.id; // Extracts '42'
  res.send(`Looking up user with ID: ${userId}`);
});
```

Query Parameters (req.query)

Meaning: Used for optional parameters like sorting, filtering, or searching. They appear at the end of a URL after a question mark (?).

Code:

```
// URL accessed by user: /search?q=apple&sort=asc
app.get('/search', (req, res) => {
  const searchQuery = req.query.q; // Extracts
  'apple'
  const sortMethod = req.query.sort; // Extracts 'asc'
  res.send(`Searching for ${searchQuery}, sorting by
  ${sortMethod}`);
});
```

5. Serving Static Files (.html, .js, .css)

By default, Express strictly protects your backend files. If a user tries to access 'style.css', the server will return a 404 Error. To allow users to download frontend files (HTML, CSS, Client-side JS, images), we must tell Express to make a specific folder "public".

Code:

```
// Put all your .html, .css, and .js files inside a folder
named 'public'
// Tell Express to serve everything inside 'public'
automatically
app.use(express.static('public'));

// Now, if you have 'public/style.css', the user can access it
at:
// http://localhost:3000/style.css
```

6. Middleware

What is Middleware?

Meaning: Middleware are functions that run **in the middle** of the Request-Response cycle. When a request comes in, it passes through middleware functions before finally reaching your route (like app.get).

Where to use it: Middleware is used for logging requests, authenticating users (checking if they are logged in), handling errors, and parsing incoming data.

Note: Every middleware function has access to `req`, `res`, and a special function called `next()`. You MUST call `next()` inside your middleware, otherwise the request will freeze and never reach the route!

Types of Middleware

1. Application-Level Middleware: Runs on EVERY single request that hits the server using `app.use()`.

```
Code:
// Global Logging Middleware
app.use((req, res, next) => {
  console.log(`New request made to: ${req.url} at ${new
Date()}`);
  next(); // Pass control to the next function/route
});
```

2. Route-Specific Middleware: Runs ONLY on specific routes.

```
Code:
// Auth Middleware
const checkAuth = (req, res, next) => {
  const isAuth = true; // Pretend we checked a token
  if (isAuth) {
    next(); // User is valid, proceed to the route
  } else {
    res.status(401).send('Unauthorized!');
  }
};

// Pass the middleware BEFORE the final (req, res) callback
app.get('/dashboard', checkAuth, (req, res) => {
  res.send('Welcome to your secret dashboard.');
```

3. Built-in Middleware: Express comes with a few built-in functions. The most important one is `express.json()`, which allows your server to understand JSON data sent in the body of a POST request.

7. Full Example: Taking Data via req.body

To handle forms or API POST requests, we need to extract data from `req.body`. Here is a complete, well-commented server demonstrating global middleware, built-in middleware, and a POST route.

Code:

```
const express = require('express');
const app = express();

// --- MIDDLEWARE ---
// 1. Built-in middleware to parse incoming JSON data
// Without this, req.body will be undefined!
app.use(express.json());

// 2. Global Custom Middleware to log requests
app.use((req, res, next) => {
  console.log(`${req.method} request made to ${req.url}`);
  next();
});

// --- ROUTES ---
// A basic GET route
app.get('/', (req, res) => {
  res.send('Welcome to the API!');
});

// A POST route to create a new user
// The client sends data like { "name": "Umar", "age": 25 }
app.post('/users', (req, res) => {
  // Extract data from the parsed body
  const userName = req.body.name;
  const userAge = req.body.age;

  // Check if the user provided the data
  if (!userName || !userAge) {
    return res.status(400).send('Error: Name and Age are required.');
```

```
// --- START SERVER ---
app.listen(3000, () => {
  console.log('Server listening on port 3000');
});
```

8. APIs and REST

What is an API?

API (Application Programming Interface) allows two applications to talk to each other. Your frontend asks the backend API for data, and the backend sends it back.

What is REST and RESTful APIs?

Meaning: REST stands for Representational State Transfer. It is a set of architectural rules and standards for building web APIs. An API that follows these rules is called a "RESTful API".

Where to use it: You use RESTful APIs whenever you need to build standard web services where a client needs to perform CRUD (Create, Read, Update, Delete) operations on a database over HTTP.

Standard HTTP Methods in REST

- GET: Retrieve data (Read). Does not change anything on the server.
- POST: Send new data to the server (Create).
- PUT: Update an existing resource completely (Update).
- PATCH: Update an existing resource partially (Update).
- DELETE: Remove a resource from the server (Delete).

9. Building a REST API (Students Array Example)

Let's build a simple Express server that performs all REST HTTP methods on an array of student data.

```
Code:
const express = require('express');
const app = express();

// Built-in middleware to parse incoming JSON data
app.use(express.json());

// Mock Database
let students = [
```

```

    { id: 1, name: 'John Doe', grade: 'A' },
    { id: 2, name: 'Jane Smith', grade: 'B' }
  ];

// 1. GET ALL (Read all students)
app.get('/students', (req, res) => {
  res.status(200).json(students);
});

// 2. GET SINGLE (Read one student by ID)
app.get('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const student = students.find(s => s.id === studentId);

  if (!student) return res.status(404).json({ message:
'Student not found' });
  res.status(200).json(student);
});

// 3. POST (Create a new student)
app.post('/students', (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name,
    grade: req.body.grade
  };
  students.push(newStudent);
  res.status(201).json({ message: 'Student created', data:
newStudent });
});

// 4. PUT (Update a student completely)
app.put('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const studentIndex = students.findIndex(s => s.id ===
studentId);

  if (studentIndex === -1) return res.status(404).json({
message: 'Student not found' });

  // Overwrite the entire object
  students[studentIndex] = { id: studentId, name:
req.body.name, grade: req.body.grade };
  res.status(200).json({ message: 'Student updated
completely', data: students[studentIndex] });
});

```

```

});

// 5. PATCH (Update a student partially)
app.patch('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const student = students.find(s => s.id === studentId);

  if (!student) return res.status(404).json({ message:
'Student not found' });

  // Update only the provided fields
  if (req.body.name) student.name = req.body.name;
  if (req.body.grade) student.grade = req.body.grade;

  res.status(200).json({ message: 'Student updated
partially', data: student });
});

// 6. DELETE (Remove a student)
app.delete('/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const initialLength = students.length;
  students = students.filter(s => s.id !== studentId);

  if (students.length === initialLength) return
res.status(404).json({ message: 'Student not found' });
  res.status(200).json({ message: 'Student deleted
successfully' });
});

app.listen(3000, () => console.log('Server running on port
3000'));

```

10. Express Router and MVC Architecture

The Problem with a Single File

If you put 50 different routes (users, products, orders, etc.) inside your main `app.js` file, it will become thousands of lines long and impossible to maintain. We need to split our code up.

MVC Architecture

Meaning: MVC stands for Model-View-Controller. It is a design pattern used to separate concerns in your application.

- Model: Handles data and database logic (e.g., Mongoose schemas).
- View: What the user sees (e.g., React, EJS, HTML).
- Controller: The "Brain". It contains the actual business logic/functions (what happens when a route is hit).

The Solution: Routers and Controllers

- Router (`express.Router()`): Used to group related routes together in a separate file (e.g., all `/students`` routes go in `studentRoutes.js``).
- Controllers: Used to take the actual callback functions (the logic) OUT of the router file, keeping the router clean. (e.g., `getStudent``, `createStudent`` go in `studentController.js``).

Full MVC Example Code

Here is how you separate the app into standard MVC files:

Code:

```
// FILE 1: controllers/studentController.js
// Purpose: Holds the actual business logic.

const getStudents = (req, res) => {
  res.status(200).json({ message: "Getting all students" });
};

const createStudent = (req, res) => {
  res.status(201).json({ message: "Student created", data:
req.body });
};

module.exports = { getStudents, createStudent };
```

Code:

```
// FILE 2: routes/studentRoutes.js
// Purpose: Maps URLs to the correct Controller functions.

const express = require('express');
const router = express.Router();
// Import the controller functions
const { getStudents, createStudent } =
require('../controllers/studentController');

// Connect the route to the controller
router.get('/', getStudents);
router.post('/', createStudent);
```

```
module.exports = router;
```

Code:

```
// FILE 3: app.js (Main Entry File)
```

```
// Purpose: Initializes the server and connects the Router.
```

```
const express = require('express');
```

```
const app = express();
```

```
// Middleware to parse JSON
```

```
app.use(express.json());
```

```
// Import our student router
```

```
const studentRoutes = require('./routes/studentRoutes');
```

```
// Use the router!
```

```
// Any request starting with '/api/students' will be sent to  
studentRoutes.js
```

```
app.use('/api/students', studentRoutes);
```

```
app.listen(3000, () => {
```

```
    console.log('MVC Server running on port 3000');
```

```
});
```